

3.5 五种页面置换算法

页面的换入换出需要磁盘I/O，会有较大的开销，因此好的页面置换算法因该追求更少的缺页率

1. 最佳置换算法 (OPT)

每次选择的页面将是以后永不使用，或者在最长时间内不再被访问的页面，这样可以保证最低的缺页率

访问页号

当要访问的页面不在内存且内存已满时，正向往后寻找最后一个出现的页号就是要淘汰的页面

“向后看”

例：假设系统为某进程分配了三个内存块，并考虑到有一下页面号引用串（会依次访问这些页面）：
7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

访问页面 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

内存块1 7 7 7 2 2 2 2 2 2 2 2 2 2 2 2 2 7 0 0

内存块2 0 0 0 0 0 0 4 0 0 0 0 0 0 0 0 0 0 0 0

内存块3 1 1 1 3 3 3 3 3 3 1 1 1 1 1 1 1 1 1 1

是否缺页 v

访问页面2时，选择从0, 1, 7中淘汰一页

按最佳置换的规则，往后寻找，最后一个出现的页号就是要淘汰的页面

整个过程缺页中断发生了9次，页面置换发生了6次

注意

缺页时未必发生页面置换

若还有可用的空闲内存块，就不用进行页面置换

最佳置换算法可以保证最低的缺页率，但实际上，只有在进程执行的过程中才能知道接下来【会访问到的是哪个页面】

操作系统无法提前预判页面访问序列。因此，最佳置换算法是【无法实现】

往后看

2. 先进先出置换算法 (FIFO)

概念

每次选择淘汰的页面是最早进入内存的页面

实现方法

把调入内存的页面根据调入的先后顺序排成一个队列，需要换出页面时选择队头页面即可

基于队列实现

队列的最大长度取决于系统为进程分配了多少个内存

只有FIFO算法才会产生Belady异常

Belady异常：分配的物理块数（驻留集）增大而页故障数（缺页次数）不减反增的异常现象

FIFO算法虽然实现简单，但是该算法与进程实际运行时的规律不适应，因为先进入的页面也有可能最近常被访问

因此，算法性能差

Belady异常

仅有它会出现该Belady异常

Cache中主存块的替换算法也有

3. 最近最久未使用置换算法 (LRU)

概念

每次淘汰的页面是最近最久未使用的页面

计组叫最近最少使用算法

两种实现方法

① 有访问字段

赋予每个页面对应的页表项中，用访问字段记录该页面自上次被访问以来所经历的时间

访问字段

当需要淘汰一个页面时，选择现有页面中t值最大的，即最近最久未使用的页面

选t最大的

适合页表项少的

② 无访问字段

操作系统维护一张页【队列】

每次选队头的元素淘汰

若某页面被访问，怎把他调到队尾

适合页表项多的

每个页面设置一个访问字段

记录页面自上次被访问以来所经历的时间，淘汰页面时选择现有页面中值【最大】的予以淘汰

页号 内存块号 状态位 访问字段 修改位 外存地址

“向前（左）看”

访问3时缺页

若需要淘汰页面，可以逆向检查此时在内存中的几个页面号。在逆向扫描过程中最后一个出现的页号就是要淘汰的页面

图示

虽然算法性能好

但是实现困难大，开销大

需要寄存器和栈（或队列）的硬件支持，实现困难开销大

往前看

比较

最佳置换算法性能最好，但无法实现

先进先出置换算法实现简单，但算法性能差

最近最久未使用算法性能好，是最接近OPT算法性能的，但实现起来需要专门的硬件支持，算法开销大

时钟置换算法是一种性能和开销较均衡的算法，又称CLOCK算法，又最近未用算法（NRU，NotRecentlyUsed）

① 【2010】简单的CLOCK置换算法

实现方法

为每个页面设置一个访问位，再将内存中的页面都通过链接指针链接成一个【循环队列】

访问位

每个页帧（页框）设置一位访问位，某页首次被装入或被访问时，其访问位被置为1

当某页被访问时，其访问位置为1

最多会【两轮】

当需要淘汰一个页面时，只需检查页的访问位（1位）

如果是0，就选择该页换出

如果是1，则将它置为0，暂不换出，继续检查下一个页面

若第一轮扫描中所有页面都是1，则将这些页面的访问位依次置为0后，再进行第二轮扫描

第二轮扫描中一定会有访问位为0的页面，因此简单CLOCK算法选择一个淘汰页面最多会经过【两轮】扫描

替换指针

当某个页面被置换时，该指针被设置指向被替换页面的下一页

简单的时钟置换算法仅考虑到一个页面最近是否被访问过

事实上，如果被淘汰的页面没有被修改过，就不需要执行I/O操作写回外存。只有被淘汰的页面被修改过时，才需要写回外存

是改进型的时钟置换算法的思想

因此，除了考虑一个页面最近是否被访问过之外，操作系统还应考虑页面有没有被修改过。【在其他条件都相同时，应优先淘汰没有被修改过的页面】，避免I/O操作

改进型CLOCK置换算法

增加修改位

修改过的页换出时要写回磁盘，反之不写回

选择页面换出时，优先考虑既未使用过又未修改过的页面（即访问位A=0，修改位M=0）

修改位=0，表示页面没有被修改过

修改位=1，表示页面被修改过

访问位A和修改位M的组合（访问位，修改位）

（0，0）

第一优先级-最近没访问，且没修改的页面

（0，1）

第二优先级-最近没访问，但修改过的页面

（1，0）

第三优先级-最近访问过，但没修改的页面

（1，1）

第四优先级-最近访问过，且修改过的页面

访问 > 修改

先选择没访问的

选择淘汰时检查两位

将所有可能被置换的页面排成一个循环队列

第一轮

从当前位置开始扫描到第一个（0，0）的帧用于替换

本轮扫描不修改任何标志位

第二轮

若第一轮扫描失败，则重新扫描，查找第一个（0，1）的帧用于替换

本轮将所有扫描过的帧【访问位】设为0

第二轮修改

第三轮

若第二轮扫描失败，则重新扫描，查找第一个（1，0）的帧用于替换

【也就是初始是（1，0）】

本轮扫描不修改任何标志位

第四轮

若第三轮扫描失败，则重新扫描，查找第一个（0，1）的帧用于替换

【也就是初始是（1，1）】

一定能找到

由于第二轮已将所有帧的访问位设为0，因此经过第三轮、第四轮扫描一定会有一个帧被选中

因此改进型CLOCK置换算法选择一个淘汰页面最多会进行【四轮】扫描

最多四轮

第二轮修改，只修改这一次

对比

共同点

都只修改访问位

都是访问位优先

不同点

简单的第一轮扫描就修改访问位

改进型的第一轮扫描不修改任何

简单最多两轮

改进型最多四轮

	算法规则	优缺点
OPT	优先淘汰最长时间内不会被访问的页面	缺页率最小，性能最好；但无法实现
FIFO	优先淘汰最先进入内存的页面	实现简单；但性能很差，可能出现Belady异常
LRU	优先淘汰最近最久没访问的页面	性能很好；但需要硬件支持，算法开销大
CLOCK（NRU）	循环扫描各页面 第一轮淘汰访问位=0的，并将扫描过的页面访问位改为1。若第一轮没选中，则进行第二轮扫描。	实现简单，算法开销小；但未考虑页面是否被修改过。
改进型CLOCK（改进型NRU）	若用（访问位，修改位）的形式表述，则 第一轮：淘汰（0，0） 第二轮：淘汰（0，1），并将扫描过的页面访问位都置为0 第三轮：淘汰（0，0） 第四轮：淘汰（0，1）	算法开销较小，性能也不错

四种算法总结